Reliable LIME under Query Budget Constraints

Qihang Feng
Department of Electrical and Computer Engineering
University of Alberta
Edmonton, Canada
qfeng5@ualberta.ca

Wonki Byun
Department of Electrical and Computer Engineering
University of Alberta
Edmonton, Canada
wonki@ualberta.ca

Zixi Chai
Department of Electrical and Computer Engineering
University of Alberta
Edmonton, Canada
zchai3@ualberta.ca

***Abstract*—LIME relies on repeated model queries, but real-world deployments often impose strict query budgets, which can make explanations unstable. In this work, we build a budget sweep pipeline with stability Jaccard, deletion-based faithfulness FaithDropK, weighted MSE fidelity, and runtime cost, enabling systematic numerical comparison under query limits. Additionally, we design a coarse-to-fine budget allocation strategy with candidate screening and focused perturbations, improving FaithDropK and highlighting key regions under lower budgets compared with original LIME. The code is available at https://github.com/QihangFeng/Reliable-LIME-under-Query-Budget-Constraints.**

Keywords—LIME, query budgets, coarse-to-fine, allocation

I. Introduction

Machine learning is widely used in real-world systems, so it is important to understand whether model predictions are trustworthy. To explain predictions from black-box classifiers, many post hoc explainers have been proposed, and LIME is one of the most widely used methods, which relies on repeated model queries to learn a simple surrogate around a specific input instance [1]. However, in real deployments, the number of allowed queries is often limited by latency and cost, and LIME explanations can become noisy and unstable under a small query budget.

In this project, we investigate the reliability of LIME for image explanations under query budget constraints and propose a budget allocation improvement. Our main contributions are summarized as follows.

- Budget sweep evaluation, a reproducible protocol that measures how LIME image explanations change under fixed query budgets, using stability Jaccard, deletion-based faithfulness FaithDropK, weighted MSE surrogate fidelity, and runtime cost.
- Coarse-to-fine budget allocation, a two-stage query allocation strategy that uses a small coarse stage to identify candidate superpixels and a fine stage to focus perturbations, improving faithfulness under limited budgets while keeping the same total number of queries.
- Qualitative evidence and trade-off analysis, an empirical study showing that the proposed allocation highlights semantically meaningful regions earlier at small budgets, while revealing trade-offs in stability and surrogate fit.

II. Background

A. Original LIME Image Workflow

This section outlines the step-by-step process of the LIME (Local Interpretable Model-agnostic Explanations) workflow for explaining image classification models.

1. Initialization and Parameter Setting

First, select the original image x to be explained and the original model f (the image classifier). Define two key parameters:

· N (Budget): The number of samples to generate.

· K: The number of superpixels to be shown

2. Superpixel Segmentation

The input image is partitioned into superpixels. A superpixel is a group of connected pixels with similar colors or textures. In the context of LIME, each superpixel is treated as a single interpretable feature.

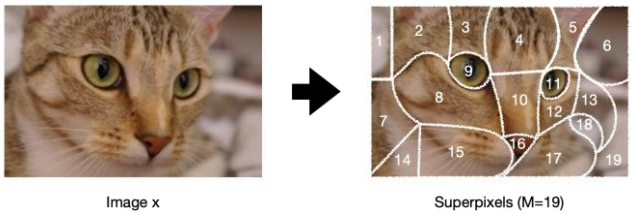

Fig. 1. Example of Superpixel Segmentation

3. Binary Mask Generation

For the M identified superpixels, a corresponding binary mask can be created. A bit value of 1 indicates the superpixel is "On" (visible) and value of 0 indicates the superpixel is "Off" (masked with gray color or blurred).

4. Sampling and Perturbation

Generate N random masks z_i and apply them to the image. These perturbed images x_i are fed into the original model f to calculate probability scores. This results in a dataset of (mask z_i, probability $f(x_i)$) pairs.

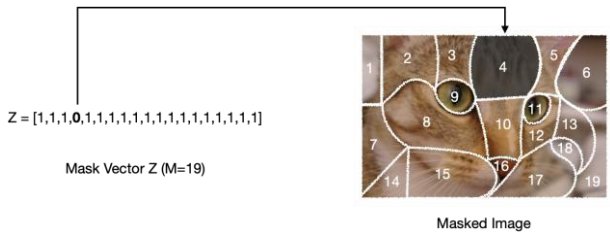

Fig. 2. Mask Vector & Masked Image

5. *Similarity Weighting*

Calculate the distance between each masked image and the original image to determine a proximity weight $\pi_x(x_i)$. This ensures that samples closer to the original image are given higher importance, while distant ones are weighted less.

6. *Training the Surrogate Model*

Using the binary masks z_i, the outputs of the original model f, and the calculated local weights $\pi_x(x_i)$, we train a local interpretable surrogate model g (such as a weighted linear regression).

$$min \sum_{\{i=1\}}^{N} \pi_x(x_i)\big(f(x_i) - g(z_i)\big)^2 + \Omega(g) \qquad (1)$$

7. *Final Explanation*

Finally, select the Top-K most significant superpixels based on the surrogate model's coefficients and highlight them on the original image to provide a visual explanation of the model's decision.

B. *Motivation*

As discussed, LIME in image classification identifies the importance of superpixels through N samples and highlights the top-K most significant features. This implies that N must be a sufficiently large number to accurately judge importance.

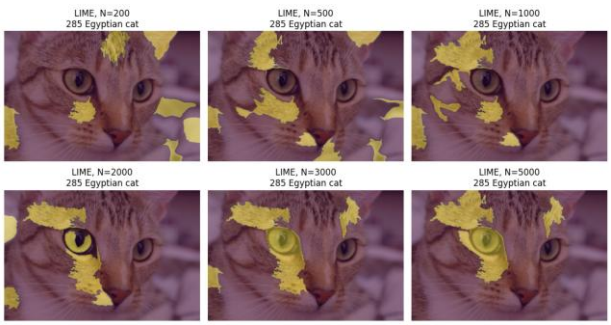

Fig. 3. LIME results using different N

Fig 3 shows the LIME results ($K = 10$) for the same image and model using different values of N. Intuitively, we can observe that when N is small, irrelevant parts like the background are highlighted. However, as N increases, the explanation narrows down to meaningful areas like the eyes or eyebrows. Looking at the numerical decrease in probability when the highlighted areas are masked, we can confirm that for N values below 3,000, the probability actually increases. This proves that our intuition is correct. Moreover, we cannot necessarily conclude that Drop-K will increase just because N increases.

TABLE I. DROP-K MEAN OF DIFFERENT N

N	200	500	1000	2000	3000	5000
Drop-K	-0.1102	-0.1257	-0.3163	-0.1635	0.1366	0.02935

While using a sufficiently large N may not be a problem in theory, reality is different. If we need to explain 100 images and the model f is so large that a single inference takes 1 second, the size of N we can practically attempt will be significantly limited. Given these finite real-world resources, if we could move beyond purely random sampling and use alternative methods to utilize the same N more effectively(identifying the important K features with a relatively lower N) we would be able to utilize LIME much more efficiently.

III. METHODOLOGY

A. *Numerical Metrics*

To evaluate LIME explanations under query budget constraints, we report four numerical metrics from four perspectives: fidelity, stability, faithfulness, and cost.

1. *Fidelity*

The post hoc explainer fits a local surrogate model g, which is a weighted linear regression, to approximate the black-box model around the input. Fidelity measures how well these surrogate predictions $\hat{y}_i$ matches the black box predictions y_i on the perturbed samples. In our implementation, we use weighted mean squared error ($WMSE$) instead of the default R^2 score.

Let $z_i \in \{0,1\}^K$ denote the i-th perturbation mask over K superpixels, and let x_i be the corresponding perturbed image. For the explained class c, we denote the black-box predicted probability by $y_i = f(x_i)_c$. The surrogate model predicts $\hat{y}_i = g(z_i)$. We use the same kernel weights as LIME, which assigns each perturbed sample a locality weight $w_i = \pi_x(z_i)$, where $\pi_x(\cdot)$ is the kernel that gives higher weight to perturbations closer to the original instance. We define the $WMSE$ as:

$$WMSE = \frac{\sum_{i=1}^{N} w_i (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} w_i} \qquad (2)$$

A smaller $WMSE$ indicates a better local surrogate fit on the sampled neighborhood.

2. *Stability*

The original LIME is stochastic because it samples perturbation masks randomly. Under the same budget N, we run the explainer multiple times with different random seeds and measure whether the selected important superpixels are consistent across runs.

For each run r, we extract the set T_r of top-k superpixels based on the explanation weights, where k matches the explanation length used in visualization and deletion tests. Given m repeated runs, stability is computed as the average pairwise Jaccard overlap:

$$Stability@k = \frac{1}{\binom{m}{2}} \sum_{1 \le i < j \le m} \frac{|T_i \cap T_j|}{|T_i \cup T_j|} \quad (3)$$

A larger value of $Stability@k$ indicates more consistent explanations across seeds.

3. Faithfulness

Faithfulness is defined as the drop in the black-box predicted probability after removing the top-k superpixels selected by the explainer. We measure it using a deletion test that directly evaluates the black-model model's sensitivity to the selected regions.

Let T_k be the set of top-k superpixels ranked by the explainer's weights, and let x_{T_k} be the image obtained by masking those superpixels using the same segmentation and masking setting. We define:

$$FaithDrop@k = f(x)_c - f(x_{T_k})_c \quad (4)$$

A larger $FaithDrop@k$ indicates that the selected superpixels are more influential for the model decision, while a smaller value indicates that the selected superpixels are less influential.

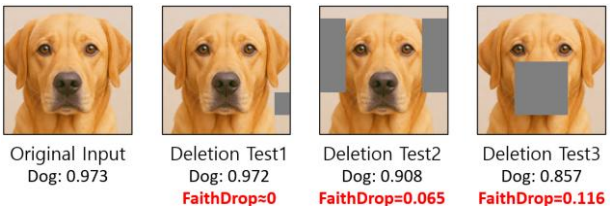

Fig. 4. An example of deletion-tests for $FaithDrop@k$

Fig 4 illustrates the deletion test. The black-box model predicts the label "dog" with probability 0.973 on the original input. In deletion test 1, the removed region is weakly related to the prediction, so the probability remains almost unchanged. In deletion test 2 and 3, removing more important regions causes larger probability drops, producing larger $FaithDrop@k$ values. This example shows that $FaithDrop@k$ provides an intuitive quantitative measure of explanation quality under a fixed query budget.

4. Cost

As for the cost, we report two measures:

- Queries, the number of black-box evaluations used to generate one explanation.
- Runtime, wall-clock time per explanations, measured in seconds.

B. Budget Sweep Pipeline

With the numerical metrics defined above, we design a budget sweep pipeline to systematically evaluate how explainers behaves under different budgets. The key idea is shown in the pseudo-code of Algorithm 1.

We define the query budget b as the number of perturbed samples generated for one explanation, which corresponds to LIME's $num_samples$. For each $b \in B$, we repeat the explanation procedure multiple times using different random seeds $s \in S$, since the perturbation masks are sampled randomly and explanation can vary across runs. We also use a constant explanation length K, which means we always select the top-K superpixels as the explanation.

Algorithm 1 Budget Sweep Evaluation with Metrics

Require: Explainer g, Image x, its interpretable version x'

Require: Budge list B, Seed list S, Explanation length K

```
M ← { }
for b ∈ B do
    E ← { }
    for s ∈ S do
        e ← g(x, x', b, s)
        T ← top_k_superpixels(e, K)
        faith ← FaithDrop(x, x', T)
        fid ← WMSE(e, x')
        E ← E ∪ < e, faith, fid >
    end for
    stability ← StabilityJaccard(E, K)
    M ← M ∪ < b, stability, E >
end for
return M
```

For each pair (b, s), we run the explainer g on the input image x to obtain an explanation e, and then extract the top-K superpixels, compute faithfulness and fidelity. After collecting results from all seeds under the same budget, we compute stability as the average Jaccard overlap of the selected superpixel sets across runs. Finally, we aggregate these values to obtain a per-budget summary.

The overall cost of this pipeline is dominated by black-box evaluations. For a fixed budget b, each explanation requires b model queries, and repeating over $|S|$ seeds results in $b|S|$ queries per budget. Therefore, the runtime scales linearly with the query budget and the number of repeated runs, which makes this evaluation suitable for analyzing the trade-off between explanation quality and query efficiency.

C. Coarse to Fine Budget Allocation

As we discussed before, when the query budget is small, vanilla LIME wastes many perturbation queries on uninformative regions, which makes the explanation noisy and less faithful. To address this, we propose a coarse-to-fine budget allocation strategy that reallocates a fixed total budget into two stages. The key idea is to spend a small portion of the budget to quickly identify potentially important superpixels, and then spend the remaining budget to refine the explanation by focusing perturbations on those candidate regions. This allows the explainer to use queries more efficiently under the same total number of black-box evaluations.

Algorithm 2 summarizes this budget allocation strategy. We first split the total budget into two parts according to an adaptive coarse ratio α, where smaller budgets allocate a relatively larger coarse fraction to reduce the risk of missing important regions, and larger budgets allocate a smaller coarse fraction since the fine stage already has sufficient capacity.

Algorithm 2 Coarse-to-Fine Budget Allocation

Require: Classifier f, Budget N
Require: Image x, its interpretable version x'
Require: Similarity kernel π_x, Explanation length K
Require: Candidate size C, Exploration ratio ε

$\alpha \leftarrow$ adaptive_coarse_ration(N)
$N_1,\ N_2 \leftarrow \alpha N,\ (1-\alpha)N$
Stage 1: Coarse
$Z_1 \leftarrow$ Coarse(f, x', π_x, N_1)
$wcoarse \leftarrow$ SparseLinearModel(Z_1, max (C, K))
$Cset \leftarrow$ get_superpixels($wcoarse$)
Stage 2: Fine
$Z_2 \leftarrow$ Fine(f, x', π_x, N_2, $Cset$, ε)
$Z \leftarrow Z_1 \cup Z_2$
$w \leftarrow$ SparseLinearModel(Z, K)
return w

Using all coarse samples Z_1, we fit a sparse linear surrogate to obtain preliminary explanation weights w_{coarse}. Then we select a candidate set C_{set} consisting of the top-C superpixels according to the magnitude of the coarse surrogate weights. Intuitively, C_{set} represents regions that are likely to be important and should receive more sampling attention in the fine stage.

Finally, we combine the sampled data from two stages and fit the final sparse linear surrogate using all collected samples. We combine Z_1 and Z_2 because the coarse stage provides broad local coverage around the instance, while the fine stage provides refined evidence around the candidate regions, and the final surrogate should learn from both.

Algorithm 2 Coarse-to-Fine Budget Allocation

$Z_1 \leftarrow$ Coarse(f, x', π_x, N_1)
$Z_1 \leftarrow \{\ \}$
for $i \in \{1,\ 2,\ 3, \dots,\ N_1\}$ **do**
  $z_i' \leftarrow$ sample_around(x')
  $Z_1 \leftarrow Z_1 \cup\ < z_i',\ f(z_i), \pi_x(z_i) >$
end for

We now describe the two stages in more detail.

Concretely, the coarse stage is similar to vanilla LIME. We sample perturbations around the original interpretable representation x' and query the black-box model to obtain outputs. For $i = 1, \dots, N_1$, we sample a perturbation mask z_i, query the black-box model, and record each training pair together with its locality weight into Z_1.

In the fine stage, we aim to refine the explanation by generating perturbations that focus on the candidate regions, while still keeping a small amount of random exploration to avoid missing important superpixels that were not selected during coarse stage. We introduce an exploration ratio ε, which controls how often we sample a random perturbation instead of a focused perturbation.

Algorithm 2 Coarse-to-Fine Budget Allocation

Stage 2: Fine
$Z_2 \leftarrow \{\ \}$
for $j \in \{1,\ 2,\ 3, \dots,\ N_2\}$ **do**
  if rand() < ε **then**
    $z_j' \leftarrow$ sample_around(x')
  else
    $z_j' \leftarrow$ sample_focused_on(Cset)
  end if
  $Z_2 \leftarrow Z_2 \cup\ < z_j',\ f(z_j), \pi_x(z_j) >$
end for

For $j = 1, \dots, N_2$, we generate a perturbation mask z_j as follows:

- $z_j' \leftarrow$ sample_around(x'): with probability ε, sample a random perturbation around x';
- $z_j' \leftarrow$ sample_focused_on(Cset): otherwise, sample a perturbation focused on the candidate set.

Each fine-stage sample is also queried through the black-box model and recorded into Z_2.

IV. Experiments and Results

A. Experiment Setup

All experiments are carried out on a single test image from the scikit-image library, namely *skimage.data.cat()*. A pre-trained ResNet-18 model [2] classifies this image as ImageNet category 285, that is, an Egyptian cat. We use the SLIC algorithm to divide the image into 100 superpixels, with a compactness of 10 and a standard deviation of 1. In order to maintain the consistency of comparison, segmentation is calculated only once and reused in all runs.

We evaluate LIME at six query budgets: $N \in \{200, 500, 1000, 2000, 3000, 5000\}$. For each budget, the experiment is repeated with 10 random seeds from 0 to 9 to capture variability. Ridge regression with regularization parameter $\alpha = 1$ is used as the surrogate model. Pairwise distances between binary perturbation masks are computed with cosine distance, and an exponential kernel with width 0.25 is used to weight the samples. Each explanation selects the 10 most important superpixels, and masked regions are filled with black.

For the coarse-to-fine method, when $N \leq 500$, 50% of the budget is allocated to the coarse stage; when $N \leq 2000$, 30% is allocated; otherwise, 20% is allocated. We set the size of the candidate set to 20, the exploration ratio to 0.2, and the batch size to 32. All other settings are consistent with the original LIME baseline.

B. Baseline Results

Table II reports the evaluation metrics of vanilla LIME across different query budgets. As the budget increases from 200 to 5000, Jaccard stability rises from 0.063 to 0.348, weighted mean squared error (WMSE) decreases from

0.000770 to 0.000163, and Drop-k faithfulness increases from -0.056 to 0.193.

TABLE II. VANILLA LIME METRICS ACROSS QUERY BUDGETS

Budget(N)	Stability (Jaccard)	Fidelity (WMSE)	Faithfulness (Drop-k)	Runtime(s)
200	0.063	0.000770	-0.056	2.74
500	0.079	0.000549	-0.019	6.80
1000	0.086	0.000343	0.023	13.85
2000	0.183	0.000267	0.155	27.93
3000	0.272	0.000223	0.145	42.10
5000	0.348	0.000163	0.193	69.71

At low budgets, especially when $N \leq 1000$, the faithfulness score is close to zero or negative. A negative Drop-k means that removing the superpixels identified by LIME as most important increases the predicted probability of the target class, so the explanation highlights the wrong regions. Stability is also very low in this range and stays below 0.1, so different random seeds lead to almost entirely different explanations. These results show that vanilla LIME is unreliable when the query budget is small.

Fig 5 shows this pattern. When $N = 200$, the highlighted area distribution is scattered and inconsistent. When $N = 5000$, the explanation focuses on the facial features of the cat, especially the eyes and nose, which are more reasonable clues for the Egyptian cat category.

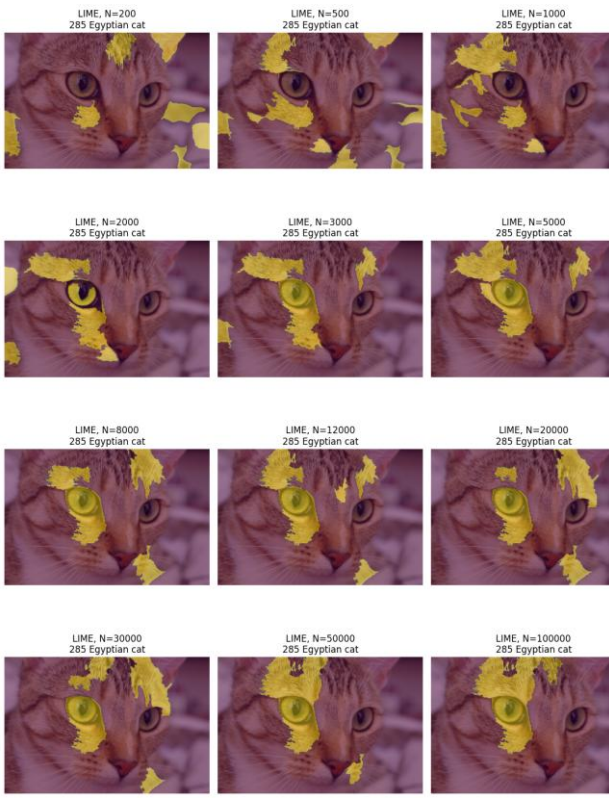

Fig. 5. Vanilla LIME explanations across query budgets

C. *Improvement Results and Discussion*

Table III summarizes the performance of the budget allocation strategy from coarse to fine under the same experimental setting as the original baseline.

TABLE III. COARSE-TO-FINE METRICS ACROSS QUERY BUDGETS

Budget(N)	Stability (Jaccard)	Fidelity (WMSE)	Faithfulness (Drop-k)	Runtime(s)
200	0.063	0.01571	0.014	2.40
500	0.090	0.02442	0.224	6.10
1000	0.100	0.02667	0.269	10.92
2000	0.153	0.02956	0.314	21.48
3000	0.143	0.02823	0.298	30.29
5000	0.177	0.03156	0.383	57.01

The clearest gain is in faithfulness. Under vanilla LIME, the Drop-k score is negative when the budget is 200 or 500. This means the explainer may highlight regions that are not truly important to the classifier. Under coarse-to-fine, the score is positive at every tested budget. At $N = 500$, it rises from -0.019 to 0.224. This suggests that the coarse stage identifies a useful candidate set and that the fine stage uses the remaining queries more effectively on regions that matter to the classifier.

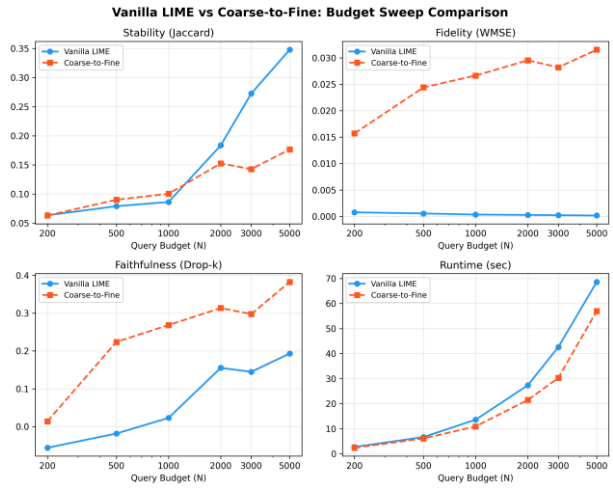

Fig. 6. Comparison of vanilla LIME and coarse-to-fine across stability, WMSE, faithfulness, and runtime

Fig 6 compares the two methods across the four evaluation metrics. Faithfulness is the main advantage of the proposed method. The gap is largest at low budgets, which matter most in practice. Even at $N = 200$ and $N = 500$, the coarse-to-fine method has produced a positive Drop-k value, while ordinary LIME still cannot identify the really influential area.

Fidelity shows the opposite pattern. Coarse-to-fine has higher weighted mean squared error (WMSE) than vanilla LIME at every tested budget. This does not necessarily mean the explanations are worse. The difference comes from the fact that the two methods sample different parts of the local neighborhood. Vanilla LIME spends many queries on neutral regions where the predicted probability remains close to zero, so a linear surrogate can fit those samples relatively easily. Coarse-to-fine concentrates more queries on sensitive regions where the black-box output changes more sharply, which makes the local regression problem harder.

Fig 7 gives an intuitive explanation of why fidelity may become worse. Neutral regions are easier for the surrogate to approximate because their contribution is small or close to zero. In contrast, influential regions are more critical and fine grained, so the black box response around them is more

complex and harder for a single linear surrogate to fit. When our method allocates more queries to these influential areas, it can improve faithfulness, but it may also make the local fitting problem more difficult and increase WMSE.

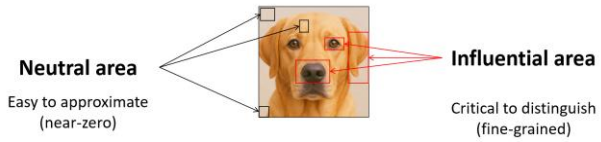

Fig. 7. Conceptual illustration of neutral regions and influential regions in local surrogate fitting

Fig 8 is consistent with this interpretation. For vanilla LIME, many sampled outputs are concentrated near zero. For coarse-to-fine, the sampled outputs cover a wider and more informative range. Coarse-to-fine therefore gives the surrogate a harder fitting problem, but it also collects samples that are more informative for explanation quality under limited budgets.

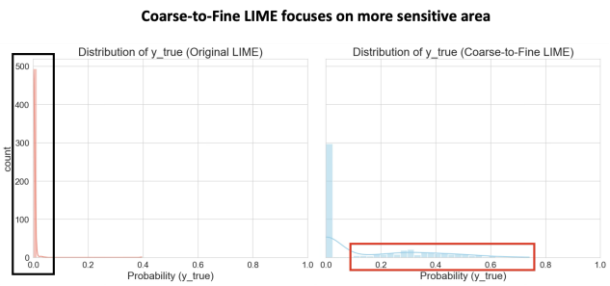

Fig. 8. Output distributions for vanilla LIME and coarse-to-fine under N = 500

Stability shows a mixed pattern. At low budgets, coarse-to-fine is slightly more stable. At $N = 500$, the Jaccard score increases from 0.079 to 0.090. At higher budgets, however, vanilla LIME becomes more stable. At $N = 5000$, vanilla LIME reaches 0.348, while coarse-to-fine remains at 0.177. One possible reason is that different random seeds produce different candidate sets in the coarse stage, and those early differences continue to affect the fine stage.

Runtime is comparable across the two methods. Since the method from coarse to fine redistributes the same total query budget instead of adding new queries, it will not introduce additional query overhead.

Under tight budgets, the proposed strategy improves what matters most: identifying truly influential regions. This improvement comes with higher WMSE and weaker stability at high budgets. Coarse-to-fine is therefore a better choice when query efficiency and faithfulness matter more than the quality of the surrogate fit.

V. Conclusion and Future Work

We studied the reliability of LIME explanations for image classification under limited query budgets. Compared with the original LIME paper, which mainly presents qualitative image examples and does not provide a systematic numerical evaluation for *lime_image* under different query budgets, our project introduces a budget-aware evaluation framework and a budget-aware improvement strategy. Specifically, our novelty lies in two aspects: first, we build a budget sweep pipeline for *lime_image* with four quantitative metrics, stability measured by Jaccard, fidelity measured by WMSE, faithfulness measured by Drop-k, and runtime, enabling controlled analysis of explanation quality as the query budget changes. Second, we propose a coarse-to-fine budget allocation strategy that reuses the same total budget more effectively by first screening candidate superpixels and then focusing the remaining queries on those candidates. The results show that vanilla LIME becomes unreliable below about $N = 1000$. Explanations vary across random seeds and may even highlight misleading regions. Our coarse-to-fine strategy improves faithfulness at every tested budget and can highlight meaningful regions with as few as 500 queries.

The improvement is most meaningful when the query budget is limited. Under large budgets, vanilla LIME can eventually identify meaningful regions, but this requires more model queries. In contrast, our coarse-to-fine strategy highlights semantically meaningful regions, such as the eye and facial area in the cat image, with fewer queries. This suggests that our method does not necessarily make LIME more powerful when enough budget is available, but it makes LIME more effective and practical when queries are costly or limited. At the same time, the results also show a trade-off, faithfulness and runtime improve, while stability and surrogate fidelity may become worse.

Future work can proceed in several directions. Because the current study uses only one test image and one model, ResNet-18, the next step is to evaluate more images and additional architectures such as VGG and Vision Transformer. We also found that coarse-to-fine does not always improve stability at high budgets, so alternative sampling strategies such as stratified sampling deserve further study. The current hyperparameters, including the coarse fraction, the exploration ratio, and the candidate set size, were chosen manually in early trials, and automatic tuning would make the method more practical. It would also be worthwhile to extend this budget-aware framework to tabular and text settings, where LIME is widely used.

Acknowledgment

We would like to thank Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin, the authors of the original LIME paper and repository, for making their work publicly available, which provided the foundation for this project. We also sincerely thank our instructor, Lei Ma, and our teaching assistant, Roy Sun, for their guidance, feedback, and support throughout this project. In addition, we appreciate the open source community for providing valuable tools, documentation, and other resources that helped us complete this work.

References

[1] Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. "" Why should i trust you?" Explaining the predictions of any classifier." In Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp. 1135-1144. 2016.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.